

Git Branching & Tools

Complete Study Guide for Technical Assessments

HackerRank / Technical Assessment Ready

#	Chapter	What You'll Learn
1	Git Fundamentals	Core concepts every engineer must know
2	Branch Mechanics	Creating, switching, deleting branches
3	Merging Strategies	Fast-forward, 3-way merge, squash
4	Rebasing	Rebase vs merge, interactive rebase
5	Remote Branches	fetch, pull, push, tracking
6	Branching Workflows	Git Flow, GitHub Flow, Trunk-Based
7	Conflict Resolution	How to handle merge conflicts
8	Advanced Tools	stash, cherry-pick, bisect, tags, reflog
9	Quick Reference	Commands at a glance
10	Practice Q&A	Test yourself before the assessment

Chapter 1 — Git Fundamentals

What is Git?

Git is a **distributed version control system (DVCS)**. Unlike older centralised systems, every developer has a **complete copy** of the entire repository — history and all — on their local machine. This means you can commit, branch, and review history entirely offline, then sync with a remote when ready.

■ Think of Git as a time machine for your code. Every commit is a snapshot you can always return to.

The Three Areas

Understanding these three areas is essential — nearly every Git operation moves code between them:

Area	Also Called	What It Contains
Working Directory	Working Tree	Files you are currently editing on disk
Staging Area	Index	Changes you have marked (git add) for the next commit
Repository	.git folder	Committed history — permanent snapshots

Core Concepts Glossary

Repository (repo) — A directory tracked by Git, containing all files and the full commit history.

Commit — A saved snapshot of staged changes. Identified by a unique SHA-1 hash (e.g. a3f2c9b).

Branch — A lightweight, movable pointer to a specific commit. Branching is nearly instant in Git.

HEAD — A special pointer that tells Git which branch (or commit) you are currently on.

Remote — A version of the repository hosted elsewhere (GitHub, GitLab, Bitbucket, etc.).

Clone — A full local copy of a remote repository, including all history.

Fork — A personal copy of someone else's repository on a hosting platform.

Tag — A named, static pointer to a specific commit — usually used for version releases.

Index / Staging — The preparation area between your working directory and the next commit.

SHA / Hash — The unique 40-character identifier for every commit (you can use the first 7 chars).

Essential Setup Commands

```
git config --global user.name 'Your Name'
git config --global user.email 'you@example.com'
git config --global core.editor 'nano' # or vim, code, etc.

git init # start a new repo in current folder
git clone <url> # download a remote repo (with full history)
git status # see what's changed / staged
git log --oneline --graph --all # visualise branch history
```

Chapter 2 — Branch Mechanics

What Is a Branch?

A branch is simply a **file containing a 40-character SHA hash** pointing to the latest commit on that line of work. This makes creating a branch essentially free — Git just writes a tiny file. There is no copying of code.

■ Rule of thumb: branch early, branch often. Branches are meant to be throwaway. If a feature fails, delete the branch and nothing is lost.

Branch Commands — Full Reference

Command	What It Does
<code>git branch</code>	List all local branches (* = current)
<code>git branch -a</code>	List local AND remote branches
<code>git branch -v</code>	List branches with last commit message
<code>git branch feature/login</code>	Create a new branch (stay on current)
<code>git switch feature/login</code>	Switch to a branch (modern syntax)
<code>git checkout feature/login</code>	Switch to a branch (classic syntax)
<code>git switch -c feature/login</code>	Create AND switch in one step (modern)
<code>git checkout -b feature/login</code>	Create AND switch in one step (classic)
<code>git branch -d feature/login</code>	Delete a merged branch (safe)
<code>git branch -D feature/login</code>	Force-delete an unmerged branch
<code>git branch -m old-name new-name</code>	Rename a branch
<code>git branch --merged</code>	Show branches already merged into HEAD
<code>git branch --no-merged</code>	Show branches NOT yet merged

How HEAD Works

HEAD is a pointer to the *currently checked-out branch*. When you commit, HEAD (and the branch it points to) moves forward to the new commit automatically.

Detached HEAD happens when you checkout a specific commit hash rather than a branch name. You can explore freely, but any commits you make are not attached to a branch and will be garbage-collected later unless you create a branch from that point.

```
git checkout a3f2c9b # enter detached HEAD (exploring old commit)
git switch -c rescue-branch # save your work by creating a branch
git switch main # return to main (exits detached HEAD)
```

■■ Never commit directly to main/master in a team project. Always work on a feature branch and merge via a pull request.

Chapter 3 — Merging Strategies

The Three Types of Merge

Fast-Forward Merge

Happens when the target branch has **no new commits** since the feature branch was created. Git simply moves the branch pointer forward — no merge commit is created. History looks perfectly linear.

```
git switch main
git merge feature/login # fast-forward if main hasn't moved
git merge --no-ff feature/login # force a merge commit even if FF possible
```

3-Way Merge (Recursive)

When both branches have diverged (each has new commits), Git finds the **common ancestor commit**, compares both branch tips against it, combines the changes, and creates a new **merge commit** with two parents. This preserves the full history of both branches.

```
git switch main
git merge feature/payments # creates a merge commit (two parents)
git log --oneline --graph # visualise the merge commit
```

Squash Merge

Collapses all commits from the feature branch into a **single new commit** on main. Keeps history clean but loses individual commit granularity. Used on GitHub as 'Squash and merge' in PRs.

```
git switch main
git merge --squash feature/cleanup
git commit -m 'Add cleanup feature' # you must commit manually after squash
```

Merge Strategy Comparison

Strategy	Merge Commit?	Preserves Feature History?	Best For
Fast-Forward	No	Yes (linear)	Simple, single-dev features
3-Way Merge	Yes	Yes (branched graph)	Team features, long-lived branches
Squash Merge	New commit	No (squashed)	Clean main history, small PRs

Chapter 4 — Rebasing

What Is Rebase?

Rebasing **replays your commits on top of another branch**, creating brand-new commits with new hashes. The result looks as if you branched off the very latest point of the target — a perfectly linear history.

```
# You are on feature/search, main has moved ahead
git switch feature/search
git rebase main # replay feature commits on top of latest main

# Then merge (will be fast-forward since history is linear)
git switch main
git merge feature/search
```

■■ **GOLDEN RULE:** Never rebase commits that have already been pushed to a shared remote branch. Rebase rewrites history — this breaks collaborators' copies.

Rebase vs Merge — Key Differences

	Merge	Rebase
History	Preserves branching graph	Creates linear history
Merge commit	Yes (3-way) or No (FF)	No — new commits created
Conflict point	Once, at merge time	Potentially once per replayed commit
Safe on public?	Yes	No — rewrites hashes
Use case	Team collaboration, PRs	Cleaning up local work before pushing

Interactive Rebase

Interactive rebase lets you **edit, reorder, squash, or drop** commits before they are pushed. This is the standard way to clean up messy local commit history.

```
git rebase -i HEAD~3 # interactively edit the last 3 commits

# In the editor you'll see:
pick a1b2c3 Add login form
pick d4e5f6 Fix typo in login
pick 7g8h9i Add login validation

# Change 'pick' to:
# squash (s) - combine with previous commit
# reword (r) - keep commit, edit message
# drop (d) - delete this commit entirely
# edit (e) - pause rebase to amend this commit
```

Chapter 5 — Remote Branches

Remote Tracking Branches

When you clone a repo, Git creates **remote-tracking references** like `origin/main`, `origin/develop`. These are read-only snapshots of where the remote branches were the last time you communicated with the server. They update automatically on `fetch/pull`.

```
git remote -v # list remotes and their URLs
git remote add origin <url> # add a remote named 'origin'
git remote rename origin upstream # rename remote
git remote remove upstream # remove a remote
```

fetch vs pull vs push

Command	What It Does	Changes Working Dir?
<code>git fetch</code>	Downloads remote changes to tracking refs only	No
<code>git fetch origin</code>	Fetch all branches from 'origin'	No
<code>git pull</code>	fetch + merge (or fetch + rebase with <code>--rebase</code> flag)	Yes
<code>git pull --rebase</code>	fetch + rebase — cleaner history, avoids merge commits	Yes
<code>git push origin main</code>	Upload local main to remote	—
<code>git push -u origin feature</code>	<code>-u</code> sets the upstream tracking link	—
<code>git push --force-with-lease</code>	Safer force push — fails if remote has new commits	—

Working With Remote Branches

```
# Check out a remote branch locally
git switch --track origin/feature/payments
# or shorthand (Git auto-detects the remote)
git switch feature/payments

# Delete a remote branch
git push origin --delete feature/old-branch

# Prune stale remote-tracking refs
git fetch --prune
git remote prune origin

# See tracking relationships
git branch -vv
```

Chapter 6 — Branching Workflows

1. Feature Branch Workflow

The simplest team workflow. Every new feature or bug fix lives on its own branch. When ready, open a **Pull Request (PR)** for review, then merge into main.

```
git switch main && git pull # start from latest main
git switch -c feature/dark-mode # create feature branch
# ... do work, commit, commit ...
git push -u origin feature/dark-mode # push to remote
# Open Pull Request on GitHub/GitLab
# After approval: merge PR, then clean up locally
git switch main && git pull
git branch -d feature/dark-mode
```

2. Git Flow

A structured model by Vincent Driessen. Uses **two permanent branches** plus several supporting branches:

Branch	Purpose
main	Production-ready code only. Tagged with version numbers.
develop	Integration branch — features merge here, not directly to main.
feature/xxx	Branched from develop; merged back to develop when done.
release/1.2.0	Branched from develop when ready to release; bug fixes only; merges to main AND develop.
hotfix/urgent-fix	Branched from main to fix critical prod bugs; merges to main AND develop.

3. GitHub Flow (Simplified)

Lighter than Git Flow. Only **one long-lived branch (main)**. Deploy directly from main after every merged PR. Ideal for teams doing continuous deployment.

```
# 6 steps of GitHub Flow:
1. Branch off main
2. Add commits
3. Open a Pull Request
4. Discuss and review
5. Deploy (test in staging if needed)
6. Merge to main
```

4. Trunk-Based Development

Everyone integrates to **main (trunk) at least once a day**. Feature branches are very short-lived (hours, not days). Requires strong automated test coverage and feature flags to hide incomplete features. Used by Google, Facebook, and most high-velocity engineering orgs.

Workflow	Branches	Release Cadence	Team Size
----------	----------	-----------------	-----------

Feature Branch	main + features	Flexible	Any
Git Flow	main+develop+more	Scheduled	Medium–Large
GitHub Flow	main + features	Continuous	Small–Medium
Trunk-Based	main only (mostly)	Continuous	Large / Fast

Chapter 7 — Conflict Resolution

When Do Conflicts Happen?

A merge conflict occurs when **two branches modify the same lines of the same file** in different ways, or when one branch deletes a file that the other branch modified. Git cannot decide which version to keep, so it pauses and asks you.

Conflict Markers — What They Mean

```
<<<<<< HEAD
const greeting = 'Hello'; // your current branch's version
=====
const greeting = 'Hi there!'; // the incoming branch's version
>>>>>> feature/new-copy
```

Edit the file to keep the correct version (or a combination of both), then remove ALL conflict markers. After that:

```
git add conflicted-file.js # mark as resolved
git commit # complete the merge
# or if you want to abort entirely:
git merge --abort
git rebase --abort
```

■ Use 'git diff' before staging to double-check your resolution. Also useful: 'git log --merge' to see which commits caused the conflict.

Prevention Strategies

- Keep branches short-lived (merge within days, not weeks).
- Pull from main frequently to stay up to date.
- Communicate with teammates about who owns which files.
- Break large files into smaller modules — fewer people touching the same code.
- Use git pull --rebase instead of plain pull to avoid unnecessary merge commits.

Chapter 8 — Advanced Git Tools

git stash — Temporary Shelf

Stash saves your uncommitted changes to a temporary stack so you can switch context without committing half-finished work.

```
git stash # stash tracked files
git stash -u # stash including untracked files
git stash push -m 'WIP login form' # stash with a label
git stash list # see all stashes
git stash pop # apply latest stash AND remove it
git stash apply stash@{2} # apply a specific stash (keep it in list)
git stash drop stash@{0} # delete a specific stash
git stash clear # delete ALL stashes
git stash branch feature/wip # create a branch from a stash
```

git cherry-pick — Pick Specific Commits

Copy one or more commits from any branch onto your current branch, creating new commits with the same changes but different hashes.

```
git cherry-pick a1b2c3d # apply one commit
git cherry-pick a1b..f6g7h8 # apply a range of commits
git cherry-pick --no-commit a1b2c3d # apply changes without committing
git cherry-pick --abort # cancel a cherry-pick in progress
```

git bisect — Find Bugs with Binary Search

Automatically find which commit introduced a bug using binary search through your history.

```
git bisect start
git bisect bad # current commit is broken
git bisect good v1.2.0 # this tag/commit was fine
# Git checks out the midpoint – you test it, then:
git bisect good # or: git bisect bad
# Repeat until Git identifies the culprit commit
git bisect reset # return to original HEAD when done
```

git reflog — Your Safety Net

Reflog records every movement of HEAD — even commits that have been 'deleted' by reset or rebase. It's how you recover from accidental disasters.

```
git reflog # see full HEAD movement history
git reflog show feature/login # reflog for a specific branch
git reset --hard HEAD@{3} # jump back to 3 moves ago
git checkout -b recovery HEAD@{5} # recover a 'lost' commit
```

git tags — Mark Releases

```
git tag # list all tags
git tag v1.0.0 # lightweight tag (just a pointer)
```

```
git tag -a v1.0.0 -m 'Release 1.0.0' # annotated tag (with message)
git push origin v1.0.0 # push a specific tag
git push origin --tags # push ALL tags
git tag -d v1.0.0 # delete a local tag
git push origin --delete v1.0.0 # delete remote tag
```

git reset — Undo Commits

Mode	Commits	Staging Area	Working Dir	Use Case
--soft	Moved back	Unchanged	Unchanged	Undo commit, keep changes staged
--mixed	Moved back	Cleared	Unchanged	Undo commit + unstage (default)
--hard	Moved back	Cleared	Cleared	Completely discard changes (dangerous!)

■■ **git reset --hard permanently deletes uncommitted changes. Prefer git revert for undoing public commits — it creates a new commit instead of rewriting history.**

Chapter 9 — Quick Reference Cheat Sheet

BRANCHES	DESCRIPTION
<code>git branch</code>	List local branches
<code>git branch -a</code>	List all branches (incl. remote)
<code>git switch -c</code>	Create + switch to new branch
<code>git switch</code>	Switch branch
<code>git branch -d</code>	Delete merged branch
<code>git branch -D</code>	Force-delete branch
<code>git branch -m old new</code>	Rename branch

MERGE & REBASE	DESCRIPTION
<code>git merge</code>	Merge branch into current
<code>git merge --no-ff</code>	Force merge commit
<code>git merge --squash</code>	Squash into single commit
<code>git rebase main</code>	Rebase current branch onto main
<code>git rebase -i HEAD~N</code>	Interactive rebase last N commits
<code>git rebase --abort</code>	Cancel rebase
<code>git rebase --continue</code>	Continue after resolving conflict

REMOTE	DESCRIPTION
<code>git remote -v</code>	List remotes
<code>git fetch</code>	Download remote changes (no merge)
<code>git pull</code>	Fetch + merge
<code>git pull --rebase</code>	Fetch + rebase
<code>git push -u origin</code>	Push + set upstream
<code>git push --force-with-lease</code>	Safer force push
<code>git push origin --delete branch</code>	Delete remote branch

STASH & CHERRY-PICK	DESCRIPTION
<code>git stash</code>	Stash current changes
<code>git stash pop</code>	Apply + remove latest stash
<code>git stash list</code>	See all stashes
<code>git cherry-pick</code>	Apply specific commit to branch

UNDO & RECOVERY	DESCRIPTION
<code>git reset --soft HEAD~1</code>	Undo last commit, keep staged

<code>git reset --hard HEAD~1</code>	Undo last commit, discard changes
<code>git revert</code>	Create undo commit (safe)
<code>git reflog</code>	See full HEAD history
<code>git bisect start/good/bad</code>	Binary search for bad commit

Chapter 10 — Practice Q&A

Read each question, formulate your answer, then reveal the answer below. These reflect the types of questions that appear in HackerRank assessments and technical interviews.

Q1. What is the difference between git fetch and git pull?

Answer: git fetch downloads remote changes into remote-tracking branches but does NOT update your working directory or local branches. git pull is essentially fetch + merge (or fetch + rebase with --rebase). Use fetch when you want to inspect changes before integrating.

Q2. What is a detached HEAD state and how do you exit it?

Answer: Detached HEAD means HEAD points directly to a commit hash rather than a branch name. You enter it with 'git checkout '. Any commits you make won't belong to a branch and may be lost. Exit safely by running 'git switch main' (or any branch), or create a new branch first with 'git switch -c my-branch'.

Q3. What is the difference between git merge --ff, --no-ff, and --squash?

Answer: --ff (fast-forward): moves branch pointer, no merge commit. --no-ff: always creates a merge commit even if FF is possible. --squash: collapses all feature commits into staged changes for one new commit, losing individual commit history.

Q4. When should you use rebase instead of merge?

Answer: Use rebase to clean up local, unpublished commits before pushing (gives linear history). Use merge for integrating shared/published branches. Never rebase commits already pushed to a shared remote — it rewrites history and breaks others' copies.

Q5. How does git cherry-pick work?

Answer: git cherry-pick copies a specific commit (by its SHA) from anywhere in history and applies its changes to your current branch as a new commit with a new hash. Useful for hotfixes: apply a fix to main without merging the entire feature branch.

Q6. What are the main branches in Git Flow?

Answer: Two permanent: main (production) and develop (integration). Supporting: feature/ (branch from develop, merge back to develop), release/* (branch from develop, merge to main AND develop), hotfix/* (branch from main, merge to main AND develop).*

Q7. How do you recover from an accidental git reset --hard?

Answer: Use git reflog to find the commit hash you were on before the reset (HEAD@{N}), then git reset --hard HEAD@{N} or git checkout -b recovery HEAD@{N} to restore it. Reflog keeps entries for ~90 days by default.

Q8. What is git stash and when would you use it?

Answer: git stash temporarily shelves uncommitted changes (both staged and unstaged) so you can switch branches without committing half-finished work. Use it when you need to context-switch urgently. Use git stash pop to restore the latest stash.

Q9. What does git bisect do?

Answer: git bisect uses binary search to find which commit introduced a bug. You mark a known bad commit (git bisect bad) and a known good commit (git bisect good). Git checks out midpoints and asks you to test each one until the culprit is found. Run 'git bisect reset' when done.

Q10. What is the difference between an annotated tag and a lightweight tag?

Answer: A lightweight tag (git tag v1.0) is just a named pointer to a commit — like a branch that doesn't move. An annotated tag (git tag -a v1.0 -m 'msg') is a full Git object with a tagger name, email, date, and message. Annotated tags are recommended for releases.

Q11. What is --force-with-lease and why is it safer than --force?

Answer: --force overwrites the remote branch no matter what. --force-with-lease checks that the remote branch hasn't been updated since you last fetched; if someone else pushed in the meantime, the push is rejected, preventing you from accidentally deleting their work.

Q12. How do merge conflicts arise and how do you resolve them?

Answer: Conflicts occur when two branches modify the same lines differently, or one branch deletes a file the other modified. Git inserts conflict markers (<<<<<<, =====, >>>>>>). You manually edit the file to the correct state, remove the markers, then run 'git add ' and 'git commit' to complete the merge.

Q13. What is the difference between git revert and git reset?

Answer: git reset moves the branch pointer backward, rewriting history — dangerous on public branches. git revert creates a NEW commit that undoes the changes of a previous commit, preserving history. Always prefer revert for undoing public/shared commits.

Q14. What is a tracking branch?

Answer: A tracking branch is a local branch configured to track a remote branch. When set up, 'git pull' and 'git push' know which remote branch to sync with automatically. Set it with 'git push -u origin ' or 'git switch --track origin'.

Q15. How would you delete a remote branch?

Answer: git push origin --delete branch-name. You can also go to GitHub/GitLab and delete it from the UI. Locally, prune stale remote-tracking refs with 'git fetch --prune'.

Good luck on your assessment! Remember: understanding the WHY behind each command is what separates candidates who pass from those who merely memorise syntax.